

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение
высшего образования «Санкт-Петербургский политехнический
университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление: 02.03.01 «Математика и компьютерные науки»

Отчет о выполнении лабораторной работы №4
по дисциплине «Математическая логика и теория формальных языков»
«Доработка компилятора языка Milan»
Вариант 1

Студент,
группы 5130201/30101

_____ Мартыненко А. П.

Преподаватель

_____ Востров А. В.

«_____» _____ 2026г.

Санкт-Петербург, 2026

Содержание

Введение	3
1 Математическое описание	4
1.1 Язык Milan	4
1.2 Функциональность цикла for в разных языках программирования	5
1.3 Грамматика языка Milan	6
1.3.1 Грамматика рекурсивного спуска	6
1.3.2 БНФ-нотация	6
1.4 Синтаксическая диаграмма	7
2 Особенности реализации	10
2.1 Добавление токенов	10
2.2 Модификация синтаксического анализатора	11
3 Результаты работы программы	16
Заключение	22
Список использованной литературы	23

Введение

В лабораторной работе необходимо:

1. Проанализировать функциональность оператора цикла в разных языках программирования.
2. Реализовать оператор цикла в компиляторе языка Milan.

```
for <переменная> := <выражение> to <выражение> step  
  <выражение> do <список операторов> od
```

3. Проработать программу на тестовых примерах.

1 Математическое описание

1.1 Язык Milan

Язык — это любое конечное подмножество цепочек символов над конечным словарем.

Язык Милан — учебный язык программирования. Программа на Милане представляет собой последовательность операторов, заключенных между ключевыми словами `begin` и `end`. Операторы отделяются друг от друга точкой с запятой. После последнего оператора в блоке точка с запятой не ставится. Компилятор `SMILAN` не учитывает регистр символов в именах переменных и ключевых словах.

В базовую версию языка Милан входят следующие конструкции: константы, идентификаторы, арифметические операции над целыми числами, операторы чтения чисел со стандартного ввода и печати чисел на стандартный вывод, оператор присваивания, условный оператор, оператор цикла с предусловием.

Компилятор `SMILAN` включает три компонента:

1. лексический анализатор;
2. синтаксический анализатор;
3. генератор команд виртуальной машины Милана.

Лексический анализатор посимвольно читает из файла текст программы и преобразует группы символов в лексемы (терминальные символы грамматики языка Милан). При этом он отслеживает номер текущей строки, который используется синтаксическим анализатором при формировании сообщений об ошибках. Также лексический анализатор удаляет пробельные символы и комментарии.

Синтаксический анализатор читает сформированную лексическим анализатором последовательность лексем и проверяет ее соответствие грамматике языка Милан. Для этого используется метод рекурсивного спуска. Если в процессе синтаксического анализа обнаруживается ошибка, анализатор формирует сообщение об ошибке, включающее сведения об ошибке и номер строки, в которой она возникла. В процессе анализа программы синтаксический анализатор генерирует набор машинных команд, соответствующие каждой конструкции языка.

Генератор кода формирует внешнее представление генерируемого кода. Генератор поддерживает буфер команд и предоставляет синтаксическому анализатору набор функций, позволяющий записать указанную команду по определенному адресу.

Все три компонента объединяются «драйвером» — управляющей программой компилятора. Драйвер анализирует аргументы командной строки,

инициализирует синтаксический анализатор и вызывает его метод `parse()`, запуская процесс трансляции.

SMILAN является однопроходным компилятором, то есть синтаксический анализ и генерация кода выполняются совместно. При этом компилятор не оптимизирует код.

1.2 Функциональность цикла `for` в разных языках программирования

Python

В Python цикл `for` — это итератор, который последовательно проходит по элементам итерируемого объекта. При итерируемом объекте `range (a, b, step)` генерируется последовательность от `a` до `b` с шагом `step`. При этом направление явно задается шагом `step`: если шаг положительный, то тело цикла выполняется, пока `start < stop`. Если шаг отрицательный, тело цикла выполняется, пока `start > stop`. Если условие не выполняется, компилятор автоматически выходит из цикла, исключая возможность бесконечного выполнения.

C++

В C++ цикл с параметром `for (init; condition; increment)` выполняет действия внутри цикла, пока условие `condition` верно. При этом знак шага `increment` не влияет на него: необходимо явно задавать направление с помощью условия. Если цикл задан некорректно (например, `for (int i = 1; i <= 5; i -= 1)`), компилятор будет бесконечно его выполнять.

Java

В Java цикл `for (init; condition; update)` действует аналогично C++, то есть направление итерации зависит от условия, задаваемого при написании программы.

Fortran

В Fortran цикл `DO переменная = start, stop, step END DO` выполняет действия в теле цикла. Знак шага `step` определяет правило конца цикла, как в Python. Цикл выполняется от `start` до `end` включительно.

Pascal

В Pascal для того, чтобы различать положительный и отрицательный шаг в цикле `for`, используются разные конструкции: для положительного шага - `for i := a to b do` (правило $i \leq b$), для отрицательного шага - `for i := a downto b do`. При этом в Pascal шаг фиксирован, то есть может быть только 1 для `to` и -1 для `downto`.

1.3 Грамматика языка Milan

1.3.1 Грамматика рекурсивного спуска

LL(k)-грамматики позволяют выполнять нисходящий синтаксический анализ, просматривая входную цепочку слева направо для построения канонического левого вывода. Здесь k указывает, сколько символов из неппрочитанной части входной цепочки используется для принятия решений. Грамматика языка Милан является LL(1) грамматикой, так как необходимо просмотреть поток всего на один символ вперед для выбора следующего правила грамматики.

Анализатор рекурсивного спуска — это вариант предсказывающего LL(1) анализа, в котором каждому нетерминалу сопоставляется рекурсивная процедура, и магазин образуется неявно при вызовах таких процедур.

Грамматика рекурсивного спуска — это такая LL(1)-грамматика, которая представляется такими синтаксическими диаграммами для каждого нетерминала, что в любом разветвлении каждой синтаксической диаграммы выбор альтернативного пути может быть сделан на основе анализа одного очередного терминального символа.

Milan является грамматикой с рекурсивным спуском, так как все правила начинаются с разных терминальных символов и отсутствует левая рекурсия.

1.3.2 БНФ-нотация

Форма Бэкуса-Наура (БНФ) — форма представления контекстно-свободных формальных грамматик, в которой правые части продукций с одинаковой левой частью объединены.

Правила построения БНФ:

- Нетерминалы помещаются в угловые скобки.
- Символ `::=` используется вместо стрелки $\rightarrow$.
- Альтернативы одного и того же нетерминала записываются в правой части одного правила через знак `|`.

Грамматика языка Милан в расширенной форме Бэкуса-Наура приведена на рисунке 1. Изменения выделены зеленым.

$$\begin{aligned}
\langle program \rangle &::= \text{'begin'} \langle statementList \rangle \text{'end'} \\
\langle statementList \rangle &::= \langle statement \rangle \text{';} \langle statementList \rangle \\
&| \epsilon \\
\langle statement \rangle &::= \langle ident \rangle \text{' := ' } \langle expression \rangle \\
&| \text{'if'} \langle relation \rangle \text{'then'} \langle statementList \rangle [\text{'else'} \langle statementList \rangle] \text{'fi'} \\
&| \text{'while'} \langle relation \rangle \text{'do'} \langle statementList \rangle \text{'od'} \\
&| \text{'write'} \text{'(' } \langle expression \rangle \text{')'} | \\
&| \text{'for'} \langle ident \rangle \text{' := ' } \langle expression \rangle \text{'to'} \langle expression \rangle \text{'step'} \langle expression \rangle \text{'do'} \\
&\quad \langle statementList \rangle \text{'od'} \\
\langle expression \rangle &::= \langle term \rangle \{ \langle addop \rangle \langle term \rangle \} \\
\langle term \rangle &::= \langle factor \rangle \{ \langle mulop \rangle \langle factor \rangle \} \\
\langle factor \rangle &::= \langle ident \rangle | \langle number \rangle | \text{'(' } \langle expression \rangle \text{')'} \\
\langle relation \rangle &::= \langle expression \rangle \langle cmp \rangle \langle expression \rangle \\
\langle addop \rangle &::= \text{'+'} | \text{'-'} \\
\langle mulop \rangle &::= \text{'*'} | \text{'/'} \\
\langle cmp \rangle &::= \text{'='} | \text{'!='} | \text{'<'} | \text{'<='} | \text{'>'} | \text{'>='} \\
\langle ident \rangle &::= \langle letter \rangle \{ \langle letter \rangle | \langle digit \rangle \} \\
\langle letter \rangle &::= \text{'a'} | \text{'b'} | \text{'c'} | \dots | \text{'z'} | \text{'A'} | \text{'B'} | \text{'C'} | \dots | \text{'Z'} \\
\langle digit \rangle &::= \text{'0'} | \text{'1'} | \text{'2'} | \text{'3'} | \text{'4'} | \text{'5'} | \text{'6'} | \text{'7'} | \text{'8'} | \text{'9'}
\end{aligned}$$

Рис. 1. Грамматика языка Милан

1.4 Синтаксическая диаграмма

На рисунке 2 представлены синтаксические диаграммы для языка Milan. Добавление семантических операций в синтаксическую диаграмму позволяет построить транслятор.

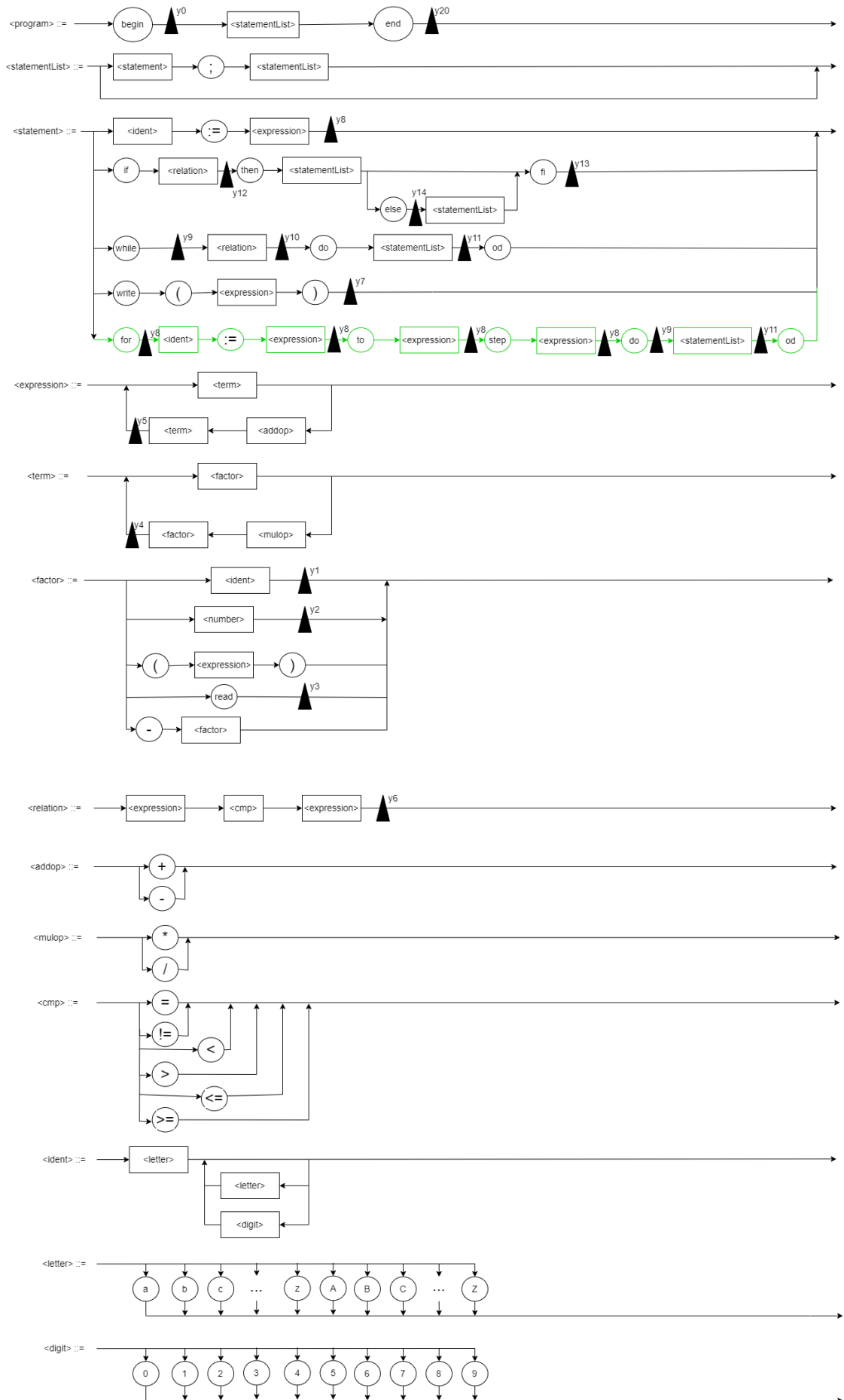


Рис. 2. Синтаксические диаграммы

Обозначения:

- C — счетчик текущей команды (адрес в памяти программ).
- $Gen(\dots)$ — процедура генерации кода виртуальной машины.

Семантические действия:

- $y0: C := 0$ (обнуление адреса текущей инструкции.)
- $y1: Gen(C : LD Nc + k); C + +$; (загрузка переменной на вершину стека)
- $y2: Gen(C : LD k); C + +$; (загрузка константы на стек)
- $y3: Gen(C : IN); C + +$; (считывание значения со стандартного ввода для read)
- $y4$: если оператор $*$, то $Gen(C : MUL)$; если $/$, то $Gen(C : DIV)$. (генерация команды умножения и деления для верхних элементов стека)
- $y5$: если оператор $+$, то $Gen(C : ADD)$; если $-$, то $Gen(C : SUB)$. (генерация команды сложения и вычитания для верхних элементов стека)
- $y6: Gen(C : CMP k); C + +$; (генерация команды сравнения двух верхних значений стека)
- $y7: Gen(C : OUT); C + +$; (генерация команды вывода значения на вершине стека)
- $y8: Gen(C : ST Nc + k); C + +$; (генерация команды сохранения значения с вершины стека по адресу переменной)
- $y9: N := C$; (сохранение адреса начала проверки условия цикла в переменную N)
- $y10: M := C; C + +$; (резервирование ячейки памяти для команды условного перехода (JMP0) и запоминание адреса в M)
- $y11: Gen(C : JMP N); C + +$; (безусловный возврат к проверке условия); $Gen(M : JMP0 C)$; (заполнение резерва: если условие ложно, выходим из цикла)
- $y12: M := C; C + +$; (резервирование места для команды JMP0)
- $y13$: если был else: $Gen(P : JMP C)$; (прыжок через блок else на код после fi); если нет $Gen(P : JMP0 P + 1)$; (если else был).
- $y14: Gen(P := C); C + +$; (резервирование в P места для команды)
- $y20: Gen(C : STOP)$ (генерация остановки виртуальной машины)

2 Особенности реализации

2.1 Добавление токенов

В файл `scanner.h` добавлены новые токены: `T_FOR`, `T_TO`, `T_STEP`.

Листинг 1. Token

```
1 enum Token {
2     T_EOF,          // Конец текстового потока
3     T_ILLEGAL,      // Признак недопустимого символа
4     T_IDENTIFIER,   // Идентификатор
5     T_NUMBER,       // Целочисленный литерал
6     T_BEGIN,        // Ключевое слово "begin"
7     T_END,          // Ключевое слово "end"
8     T_IF,           // Ключевое слово "if"
9     T_THEN,         // Ключевое слово "then"
10    T_ELSE,          // Ключевое слово "else"
11    T_FI,            // Ключевое слово "fi"
12    T_WHILE,         // Ключевое слово "while"
13    T_DO,            // Ключевое слово "do"
14    T_OD,            // Ключевое слово "od"
15    T_WRITE,         // Ключевое слово "write"
16    T_READ,          // Ключевое слово "read"
17    T_ASSIGN,        // Оператор ":@"
18    T_ADDOP,         // Сводная лексема для "+" и "-" операция( типа сложения)
19    T_MULOP,         // Сводная лексема для "*" и "/" операция( типа умножения)
20    T_CMP,           // Сводная лексема для операторов отношения
21    T_LPAREN,        // Открывающая скобка
22    T_RPAREN,        // Закрывающая скобка
23    T_SEMICOLON,     // ";"
24    // добавления
25    T_FOR,
26    T_TO,
27    T_STEP
28 };
29
30
```

В конструктор класса `Scanner` добавлена регистрация ключевых слов:

Листинг 2. Scanner()

```
1 explicit Scanner(const string& fileName, istream& input)
2 : fileName_(fileName), lineNumber_(1), input_(input)
3 {
4     keywords_["begin"] = T_BEGIN;
5     keywords_["end"] = T_END;
6     keywords_["if"] = T_IF;
7     keywords_["then"] = T_THEN;
8     keywords_["else"] = T_ELSE;
9     keywords_["fi"] = T_FI;
10    keywords_["while"] = T_WHILE;
11    keywords_["do"] = T_DO;
12    keywords_["od"] = T_OD;
13    keywords_["write"] = T_WRITE;
14    keywords_["read"] = T_READ;
15    // дополнения
16    keywords_["for"] = T_FOR;
17    keywords_["to"] = T_TO;
```

```

18 keywords_["step"] = T_STEP;
19
20 nextChar();
21 }

```

В файл `scanner.cpp` в массиве `tokenNames[]` были добавлены представления новых токенов как строк:

Листинг 3. `tokenNames`

```

1 static const char * tokenNames_[] = {
2     "end of file",
3     "illegal token",
4     "identifier",
5     "number",
6     "'BEGIN'",
7     "'END'",
8     "'IF'",
9     "'THEN'",
10    "'ELSE'",
11    "'FI'",
12    "'WHILE'",
13    "'DO'",
14    "'OD'",
15    "'WRITE'",
16    "'READ'",
17    "':='",
18    "'+' or '-'",
19    "'*' or '/'",
20    "comparison operator",
21    "'(",
22    "')'",
23    "';'",
24    // добавления
25    "'FOR'",
26    "'TO'",
27    "'STEP'"
28 };
29

```

Новые строки обрабатываются в рамках `nextToken()` как лексемы, соответствующие ключевым словам.

2.2 Модификация синтаксического анализатора

Для добавления логики цикла `for` был изменен метод `statement` класса `Parser`.

Если очередная лексема равна `T_FOR`, синтаксический анализатор переходит в ветку цикла `for`. Следующая после `for` лексема должна быть идентификатором, которую анализатор добавляет в таблицу имен. После этого должен следовать знак присваивания `T_ASSIGN`. За `:=` должно следовать арифметическое выражение, поэтому вызывается метод `expression()`. В результате выполнения этой последовательности на вершине стека будет находиться значение выражения. Чтобы выполнить присваивание, достаточно записать значение с вершины стека в память по адресу переменной

в левой части оператора присваивания с помощью инструкции `STORE`, аргументом которой будет запомненный ранее адрес переменной начального значения (`varAddr`). Далее подобным образом проверяем наличие `T_TO`, сохраняем также с помощью `expression()` переменную конца интервала, проверяем наличие `T_STEP`, сохраняем переменную шага.

Далее проверяется, что следующая лексема равна `T_DO` и запоминается адрес начала проверки условия. Проверяется знак шага: если он отрицательный, анализатор должен перейти на проверку для отрицательного шага. Так как этот адрес еще не известен, анализатор резервирует адрес для условного перехода `JUMP_NO` и запоминает адрес этой инструкции в переменной `jumpToNeg`. Для положительного шага проверяется условие `var ≤ end`. Если условие ложно, то анализатор резервирует адреса для команд перехода - выхода из цикла `jumpExit` и пропуска проверки отрицательного шага. Далее идет блок проверки для отрицательного шага, адрес которого заранее устанавливается с помощью `emitAt` и переменной `jumpToNeg`. Для отрицательного шага проверяется условие `var ≥ end`. Если условие ложно, анализатор резервирует адрес для команды перехода `jumpExitNeg`.

После проверок шага анализатор генерирует `JUMP` - безусловный переход к телу цикла. Он вызывает метод `statementList()` для обработки тела цикла. После выполнения тела счетчик увеличивается на значение шага: загружаются два значения, суммируются и сохраняются в `varAddr`.

Затем анализатор генерирует инструкцию безусловного перехода к началу проверки условия и записывает в зарезервированную ячейку инструкцию условного перехода по известному теперь адресу конца цикла.

После этого записываем адрес, который является выходом из цикла. Адреса, зарезервированные до этого для выхода (`jumpExit`, `jumpExitNeg`), заполняются этим адресом.

Для завершения анализа ожидается, что очередная лексема равна ожидаемому значению `T_OD`. Структура исполняемого кода приведена в листинге 4.

Вход: токен из сканера (`scanner_`).

Выход: генерация байт-кода для создания команды цикла с параметром.

Листинг 4. `statement`

```
1 void Parser::statement()
2 {
3     // Если встречаем переменную, то запоминаем ее адрес или добавляем новую если не
    встретили.
4     // Следующей лексемой должно быть присваивание. Затем идет блок expression,
    который возвращает значение на вершину стека.
5     // Записываем это значение по адресу нашей переменной
6     if(see(T_IDENTIFIER)) {
7         int varAddress = findOrAddVariable(scanner_>getStringValue());
8         next();
9         mustBe(T_ASSIGN);
10        expression();
```

```

11     codegen_ ->emit(STORE, varAddress);
12 }
13 // Если встретили IF, то затем должно следовать условие. На вершине стека лежит 1
14 // или 0 в зависимости от выполнения условия.
15 // Затем зарезервируем место для условного перехода JUMP_NO к блоку ELSE переход(
16 // в случае ложного условия). Адрес перехода
17 // станет известным только после того, как будет сгенерирован код для блока THEN.
18 else if(match(T_IF)) {
19     relation();
20
21     int jumpNoAddress = codegen_ ->reserve();
22
23     mustBe(T_THEN);
24     statementList();
25     if(match(T_ELSE)) {
26         //Если есть блок ELSE, то чтобы не выполнять его в случае выполнения THEN,
27         //зарезервируем место для команды JUMP в конец этого блока
28         int jumpAddress = codegen_ ->reserve();
29         //Заполним зарезервированное место после проверки условия инструкцией перехода
30         //в начало блока ELSE.
31         codegen_ ->emitAt(jumpNoAddress, JUMP_NO, codegen_ ->getCurrentAddress
32         ());
33         statementList();
34         //Заполним второй адрес инструкцией перехода в конец условного блока ELSE.
35         codegen_ ->emitAt(jumpAddress, JUMP, codegen_ ->getCurrentAddress());
36     }
37     else {
38         //Если блок ELSE отсутствует, то в зарезервированный адрес после проверки
39         //условия будет записана
40         //инструкция условного перехода в конец оператора IF...THEN
41         codegen_ ->emitAt(jumpNoAddress, JUMP_NO, codegen_ ->getCurrentAddress
42         ());
43     }
44
45     mustBe(T_FI);
46 }
47
48 else if(match(T_WHILE)) {
49     //запоминаем адрес начала проверки условия.
50     int conditionAddress = codegen_ ->getCurrentAddress();
51     relation();
52     //резервируем место под инструкцию условного перехода для выхода из цикла.
53     int jumpNoAddress = codegen_ ->reserve();
54     mustBe(T_DO);
55     statementList();
56     mustBe(T_OD);
57     //переходим по адресу проверки условия
58     codegen_ ->emit(JUMP, conditionAddress);
59     //заполняем зарезервированный адрес инструкцией условного перехода на следующий
60     //за циклом оператор.
61     codegen_ ->emitAt(jumpNoAddress, JUMP_NO, codegen_ ->getCurrentAddress()
62     );
63 }
64 else if(match(T_WRITE)) {
65     mustBe(T_LPAREN);
66     expression();
67     mustBe(T_HPAREN);
68     codegen_ ->emit(PRINT);
69 }
70 //обновления

```

```

63 else if (match(T_FOR)) {
64     // for <ident> := <expression> to <expression> step <expression> do <
statementList> od
65     if (!see(T_IDENTIFIER)) {
66         reportError("identifier expected after 'for'");
67         recover(T_DO);
68         return;
69     }
70     int varAddr = findOrAddVariable(scanner_ ->getStringValue());
71     next();
72
73     mustBe(T_ASSIGN);
74     expression();
75     codegen_ ->emit(STORE, varAddr);
76
77     mustBe(T_TO);
78     // сохраняем end во временную переменную
79     int endAddr = lastVar_++;
80     expression();
81     codegen_ ->emit(STORE, endAddr);
82     // сохраняем step во временную переменную
83     mustBe(T_STEP);
84     int stepAddr = lastVar_++;
85     expression();
86     codegen_ ->emit(STORE, stepAddr);
87
88     mustBe(T_DO);
89
90     // начало цикла
91     int conditionAddr = codegen_ ->getCurrentAddress();
92     codegen_ ->emit(LOAD, stepAddr);
93     codegen_ ->emit(PUSH, 0);
94     codegen_ ->emit(COMPARE, 3); // >
95     int jumpToNeg = codegen_ ->reserve(); //если отрицательный step
96
97     // step > 0
98     codegen_ ->emit(LOAD, varAddr);
99     codegen_ ->emit(LOAD, endAddr);
100    codegen_ ->emit(COMPARE, 4); // var <= end
101    // выходим, если step>end
102    int jumpExit = codegen_ ->reserve();
103    // если условие выполнено, пропускаем step<0
104    int jumpSkip = codegen_ ->reserve();
105
106    // step < 0
107    codegen_ ->emitAt(jumpToNeg, JUMP_NO, codegen_ ->getCurrentAddress());
108    codegen_ ->emit(LOAD, varAddr);
109    codegen_ ->emit(LOAD, endAddr);
110    codegen_ ->emit(COMPARE, 5); // var >= end
111
112    int jumpExitNeg = codegen_ ->reserve(); //если var < limit
113    // идем в цикл
114    codegen_ ->emitAt(jumpSkip, JUMP, codegen_ ->getCurrentAddress());
115
116    // тело цикла
117    statementList();
118    // i += step
119    codegen_ ->emit(LOAD, varAddr);
120    codegen_ ->emit(LOAD, stepAddr);
121    codegen_ ->emit(ADD);

```

```

122     codegen_ -> emit(STORE, varAddr);
123
124     // снова проверяем условия
125     codegen_ -> emit(JUMP, conditionAddr);
126
127     // или идем на выход
128     int exitAddr = codegen_ -> getCurrentAddress();
129     codegen_ -> emitAt(jumpExit, JUMP_NO, exitAddr);
130     codegen_ -> emitAt(jumpExitNeg, JUMP_NO, exitAddr);
131
132     mustBe(T_OD);
133 }
134 else {
135     reportError("statement expected.");
136 }
137 }

```

3 Результаты работы программы

Примеры работы программы:

Р-код

1. Простой цикл

```
BEGIN
n := READ;
p := READ;
st := READ;
FOR i := n TO p STEP st DO
WRITE(i)
OD
END
```

```
0: INPUT
1: STORE 0
2: INPUT
3: STORE 1
4: INPUT
5: STORE 2
6: LOAD 0
7: STORE 3
8: LOAD 1
9: STORE 4
10: LOAD 2
11: STORE 5
12: LOAD 5
13: PUSH 0
14: COMPARE 3
15: JUMP_NO 21
16: LOAD 3
17: LOAD 4
18: COMPARE 4
19: JUMP_NO 32
20: JUMP 25
21: LOAD 3
22: LOAD 4
23: COMPARE 5
24: JUMP_NO 32
25: LOAD 3
26: PRINT
27: LOAD 3
28: LOAD 5
29: ADD
30: STORE 3
31: JUMP 12
32: STOP
```

Результаты выполнения представлены на рисунках 3- 7.

```
Reading input from mybaseloop.out
> 1
> 5
> 1
1
2
3
4
5
```

Рис. 3. Цикл идет по числам от 1 до 5 с шагом 1

```
Reading input from mybaseloop.out
> 1
> 10
> 2
1
3
5
7
9
```

Рис. 4. Цикл идет по числам от 1 до 10 с шагом 2

```
Reading input from mybaseloop.out
> 7
> 3
> -1
7
6
5
4
3
```

Рис. 5. Цикл идет по числам от 7 до 3 с шагом -1

```
Reading input from mybaseloop.out
> 5
> 1
> 1
```

Рис. 6. Цикл идет по числам от 5 до 1 с шагом 1

```
Reading input from mybaseloop.out
> s
Error: illegal input
Code:

      0      INPUT
VM error 17
```

Рис. 7. Некорректный ввод

**2. Цикл, который считает
сумму четных чисел от 0 до n
(n > 0)**

```
BEGIN
n := READ;
start := 0;
st := 2;
sum := 0;
FOR i := start TO n STEP st DO
sum := sum + i
OD;
WRITE(sum)
END
```

```
0: INPUT
1: STORE 0
2: PUSH 0
3: STORE 1
4: PUSH 2
5: STORE 2
6: PUSH 0
7: STORE 3
8: LOAD 1
9: STORE 4
10: LOAD 0
11: STORE 5
12: LOAD 2
13: STORE 6
14: LOAD 6
15: PUSH 0
16: COMPARE 3
17: JUMP_NO 23
18: LOAD 4
19: LOAD 5
20: COMPARE 4
21: JUMP_NO 36
22: JUMP 27
23: LOAD 4
24: LOAD 5
25: COMPARE 5
26: JUMP_NO 36
27: LOAD 3
28: LOAD 4
29: ADD
30: STORE 3
31: LOAD 4
32: LOAD 6
33: ADD
34: STORE 4
35: JUMP 14
36: LOAD 3
37: PRINT
38: STOP
```

```
Reading input from mycomplexloop.out
> 10
30
```

Рис. 8. Цикл считает сумму четных чисел до 10 ($2+4+6+8+10$)

```
Reading input from mycomplexloop.out
> 7
12
```

Рис. 9. Цикл считает сумму четных чисел до 7 ($2+4+6$)

```
Reading input from mycomplexloop.out
> -5
0
```

Рис. 10. Ввод при $n < 0$

3. Некорректная программа: нет ключевого слова DO

```
BEGIN
n := READ;
p := READ;
st := READ;
FOR i := n TO p STEP st
WRITE(i)
OD
END
```

Результат компиляции:

```
Line 6: 'WRITE' found while 'DO' expected.
Line 9: statement expected.
Line 9: end of file found while 'OD' expected.
Line 9: end of file found while 'END' expected.
```

Р-код (часть 1)

```

0:  PUSH    1
1:  STORE   0
2:  PUSH    4
3:  STORE   1
4:  PUSH    1
5:  STORE   2
6:  LOAD    0
7:  STORE   3
8:  LOAD    1
9:  STORE   4
10: LOAD    2
11: STORE   5
12: LOAD    5
13: PUSH    0
14: COMPARE 3
15: JUMP_NO 21
16: LOAD    3
17: LOAD    4
18: COMPARE 4
19: JUMP_NO 58
20: JUMP     25
21: LOAD    3
22: LOAD    4
23: COMPARE 5
24: JUMP_NO 58
25: LOAD    0
26: STORE   6
27: LOAD    1
28: STORE   7
29: LOAD    2
30: STORE   8
31: LOAD    8
32: PUSH    0
33: COMPARE 3
34: JUMP_NO 40
35: LOAD    6
36: LOAD    7
37: COMPARE 4
38: JUMP_NO 53

```

4. Цикл, который выводит результаты умножения всех перестановок $i*j$

```

BEGIN
n := 1;
m := 4;
st := 1;
FOR i := n TO m STEP st DO
FOR j := n TO m STEP st DO
WRITE(i*j)
OD
OD
END

```

Р-код (часть 2)

```

39: JUMP     44
40: LOAD     6
41: LOAD     7
42: COMPARE  5
43: JUMP_NO  53
44: LOAD     3
45: LOAD     6
46: MULT
47: PRINT
48: LOAD     6
49: LOAD     8
50: ADD
51: STORE     6
52: JUMP     31
53: LOAD     3
54: LOAD     5
55: ADD
56: STORE     3
57: JUMP     12
58: STOP

```

```
Reading input from mynestedloop.out
1
2
3
4
2
4
6
8
3
6
9
12
4
8
12
16
```

Рис. 11. Программа выводит результаты умножения $i*j$ при заданных i, j

Заключение

В ходе выполнения лабораторной работы была успешно реализована доработка компилятора языка Milan, которая реализует оператор цикла со счетчиком. Для этого была расширена грамматика языка в нотации БНФ и построены соответствующие синтаксические диаграммы. Реализованная программа была проверена на наборе примеров, включающем как корректные, так и некорректные входные данные.

Достоинства программы:

- гарантируется семантическая целостность оператора;
- цикл учитывает возможность как положительного, так и отрицательного шага;
- минимальные изменения в существующую архитектуру компилятора, сохранение всех ранее реализованных функций.

Недостатки программы:

- временные переменные `endAddr` и `stepAddr` занимают ячейки памяти на всё время выполнения цикла;
- отсутствует оптимизация при генерации команд виртуальной машины.

Масштабирование программы:

- удаление локального счетчика `varAddr` после обработки `OD`;
- оптимизация генерируемого кода (кэширование переменных при частом использовании).

Список использованной литературы

- [1] Секция «Телематика». — Текст : электронный // tema.spbstu.ru : [сайт]. URL: <https://tema.spbstu.ru/mathem/> (дата обращения: 13.04.2026).
- [2] Сети, Р.; Ахо, А. Компиляторы: принципы, технологии и инструментарий - М.: Издательство «Наука», 2008. - 1178 с. (дата обращения: 13.04.2026).
- [3] Э. Ф. Аллахвердиев, Д. А. Тимофеев Компилятор CMilan — Текст : электронный // <https://github.com> : [сайт]. URL: <https://github.com/EgorGrisina/MyCompiler/> (дата обращения: 13.04.2026).